

Drone Airspace Guardian — Hackathon Roadmap

2026-09-18 · @Someone

The problem, in plain terms

Airports, hospitals, cities and disaster zones will soon have many drones flying at once, with nobody managing that shared airspace the way air traffic control manages planes. Your job for this track is to build a small air-traffic-control system for drones.

Concretely, the platform needs to:

1. Show a live map with several drones moving on it.
2. Let someone draw a restricted "no-fly" zone (an airport, a hospital, a temporary danger area).
3. Detect when a drone is about to enter a restricted zone or get too close to another drone.
4. Automatically calculate a new, safe route around the problem and reroute the drone.
5. Give each drone a "mission" — a start point, an end point, and a path between them.
6. Alert a human operator when something dangerous happens.

MVP, in one sentence: a live map with a handful of simulated drones flying their missions, one no-fly zone, and a button that triggers an emergency ("helicopter entering airspace") that automatically detects which drones are affected and reroutes them in real time, with a visible alert. Everything past that sentence — predictive maintenance, real flight-data realism, visual polish — is a layer on top, not the core. If the core sentence isn't working, nothing else matters for review 1.

The killer demo moment: 10 drones flying calmly, then you click "Emergency: helicopter inbound." The system detects which drones are in the danger radius, redraws their paths live on the map, and fires a red alert banner — all within a couple of seconds, with no manual intervention. That single moment should be rehearsed and protected above everything else you build.

Team roles — who owns what

Three of you, three layers of the system. Whichever of you takes which layer, keep the boundaries clean so you can work in parallel without stepping on each other's code.

Layer	Owns	Main tools
Backend & simulation	The drone simulation loop, the API, real-time position updates, the reroute/pathfinding logic	Python (FastAPI) or Node (Express + Socket.IO)
Frontend & map	The live map, drone markers, no-fly zone drawing, the emergency button, alert banners, animation	Next.js + Mapbox GL JS (or Leaflet as a fallback)
AI/ML & data	Training the predictive-maintenance model on the C-MAPSS data, wiring in the OpenSky trajectories, collision-detection logic, gluing frontend to backend	Python, scikit-learn, pandas

Suggested split: Rohit takes AI/ML & data — it fits the AI & Data Engineering coursework, and it means one person owns "why is this AI-powered" when judges ask. Pranav and Joel split backend and frontend between them; let whoever is stronger in each area self-select rather than being assigned, since there's no time to fight friction.

One of you should also own the pitch: keeping the demo script current, rehearsing the walkthrough, and being ready to explain the whole architecture to judges even for pieces you didn't personally write. All three should be able to explain the full pipeline at a basic level — in an elimination round, "that's not my part" is a bad answer.

Hour-by-hour timeline

Built backward from your three checkpoints: review 1 (5:00–8:00 PM today), review 2 (1:00–4:00 AM tonight), review 3 — final (10:00 AM–1:00 PM tomorrow).

Time	Block	Goal
11:00 AM – 12:00 PM	Setup	Repo created, folders agreed, each person's dev environment running, map API key obtained

Time	Block	Goal
12:00 – 2:30 PM	Core build	Backend: simulation loop + WebSocket. Frontend: map with moving dummy markers. AI: RUL model trained on C-MAPSS
2:30 – 4:00 PM	Integration	Frontend connects to backend's live feed; no-fly zone drawing works; emergency button triggers a detectable event
4:00 – 4:45 PM	Reroute logic + alert	Backend computes a detour path around the zone; frontend animates it and shows a red alert banner
4:45 – 5:00 PM	Freeze & rehearse	Stop building new features; run the demo twice end to end
5:00 – 8:00 PM	Review 1 (elimination)	Present the MVP: map, missions, one no-fly zone, one emergency reroute
8:00 – 9:00 PM	Debrief + reprioritize	Note judge feedback; decide what round 2 actually needs (see MVP checklist)
9:00 – 11:30 PM	Round 2 build	Predictive-maintenance gauge per drone (RUL model live), OpenSky-based realistic drone paths, multiple no-fly zones
11:30 PM – 12:30 AM	Integration + polish	Wire the health gauge into rerouting (a low-health drone gets sent to land, not just detoured); tidy the UI
12:30 – 1:00 AM	Rehearse	Run the demo end to end at least twice
1:00 – 4:00 AM	Review 2 (elimination)	Present the expanded system
4:00 – 8:00 AM	Rest, rotating	Sleep in shifts — e.g. one person awake at all times to fix anything that breaks overnight, the other two sleep 2 hours each
8:00 – 9:30 AM	Final polish	Fix anything fragile, tighten visuals, record a backup demo video
9:30 – 10:00 AM	Final rehearsal	Full run-through, assign who says what

Time	Block	Goal
10:00 AM – 1:00 PM	Review 3 (final)	Full presentation and demo

Protect the 4:45 PM and 12:30 AM rehearsal slots above almost everything else — a team that has run its demo twice beats a team that added one more feature at the last minute.

Architecture, explained from zero

Four pieces, each doing one job:

1. **Simulation engine** (backend) — a loop that runs every second or two, moving each drone a little further along its mission path, and checking for danger. This is the "brain."
2. **Real-time channel** (WebSocket) — a live pipe from backend to frontend so drone positions update on the map without the browser having to ask for them repeatedly. Think of it as a phone line that stays open, versus refreshing a page.
3. **Map UI** (frontend) — draws drones, zones, and routes, and lets a human draw a no-fly zone or press "emergency."
4. **State** — where drone positions, missions and zones live while the app runs. For a 28-hour build, keep this in memory in the backend process (a Python dict or list) rather than standing up PostgreSQL — a database adds setup time and a failure point you don't need for a live simulation. Only add SQLite later, and only if you need to save/replay a scenario for the demo.

flowchart LR

```

SIM[Simulation engine] -- positions every tick --> WS[WebSocket]
WS -- live updates --> MAP[Map UI]
MAP -- draw zone / emergency --> API[REST API]
API --> SIM
SIM -- checks --> DETECT[Conflict detector]
DETECT -- affected drones --> REROUTE[Reroute calculator]
REROUTE -- new path --> SIM
ML[RUL model] -- health score --> SIM

```

How the reroute logic works, simplest version: for each drone, measure the distance from its next few path points to the danger zone's center. If that distance is less than the zone's radius, the drone is "affected." For an affected drone, calculate a new waypoint offset to the side of the zone (basic geometry — push the path outward, perpendicular to the line

between the drone and the zone center) and send the drone there before resuming its original mission. This is enough for a strong demo; a full A* grid search is a stretch goal only if round 1 is solid and you have spare time before round 2.

Why WebSocket and not just refreshing: judges will notice if the map only updates when you reload the page. Socket.IO (Node) or FastAPI's native WebSocket support (Python) both give you this in a few lines — ask your AI assistant for a minimal working example in whichever language your backend person is more comfortable with, and get that connection working in the first hour, before building features on top of it.

Using your datasets — and what to skip

You have two usable datasets already downloaded, plus two very large ones you haven't downloaded (VisDrone detection/tracking, aerial segmentation). Given the time budget, don't download or touch those two for review 1 or 2 — they need real computer-vision training pipelines, which is hours you don't have. If round 1 and 2 go well and you have spare time before the final, a static demo (running a pretrained detector on a couple of sample images, not training your own) is the only realistic way to touch them at all.

1. NASA C-MAPSS (`train/test/RUL_FD00X.txt`) — predictive maintenance. This is turbofan jet-engine sensor data: each row is one operational cycle for one engine, with 21 sensor readings and 3 settings, run until the engine fails. The track brief explicitly calls out "predictive maintenance" as a target area, so reuse this dataset as your drone fleet's health data — pretend each simulated drone has a "motor/battery health" signal that degrades like these engines do. Train a simple regression model (scikit-learn's `RandomForestRegressor` or even linear regression) to predict Remaining Useful Life (RUL) — how many more cycles before failure — from the sensor columns. This trains in minutes on a laptop; you don't need deep learning here. In the app, this becomes a "predicted flight time remaining" gauge per drone, and a drone whose predicted RUL drops below a threshold should get automatically flagged and rerouted to land, exactly like a no-fly-zone reroute.

2. OpenSky ADS-B trajectories (`opensky_trajectories.csv`) — realistic movement. This is real aircraft telemetry (lat/lon/altitude/speed/heading) over the Indian subcontinent, about 180 aircraft across 20 timesteps. Use it to seed realistic-looking drone paths instead of pure random-walk movement — pull a handful of real trajectories, scale or reposition them onto your demo map area if needed, and replay them as your

simulated drones' missions. This is a fast, low-risk way to make the live map look far more convincing than dots moving in straight lines, and it's a genuine "real-world data" talking point for judges.

Don't over-invest in either dataset before the core MVP (map, zones, reroute, alert) works — both are enhancements, not the thing that gets you through review 1.

AI, in two senses — how you build with it, and where it lives in the product

Using AI to build fast, as a 3-person basic-level team. Don't share one AI chat session across all three of you — it gets confused about whose code it's looking at and answers drift. Instead:

- Each person runs their own AI coding session (Claude, ChatGPT, Copilot — whatever each of you is comfortable with) scoped to their own layer: backend person's session only ever talks about the backend folder, frontend person's only about the frontend folder, and so on.
- Start each session by pasting in this roadmap's relevant section plus the exact file(s) you're working on, so the AI has the same context you do — don't make it guess your architecture.
- When you hit a wall for more than 10–15 minutes, ask your AI for the smallest possible working version first ("give me the simplest version that runs"), then improve it — don't ask for the polished version up front, you don't have time to debug something you don't understand.
- Before merging anything into `main`, skim what the AI wrote well enough to explain it in one sentence — you'll need to defend it to judges.

Where AI actually lives inside the app. The track brief rewards real AI/ML use, not just the word "AI" in your pitch, so make sure at least one thing in your demo is a genuinely trained model, not just hardcoded rules:

- **Predictive maintenance (real ML):** the RUL regression model trained on C-MAPSS (see the datasets section) is your anchor AI feature — it's an actual trained model making an actual prediction, which is the strongest "AI" claim you have.

- **Collision/zone detection (rules, and that's fine):** distance-based geometry checks ("is this drone within X meters of the zone center") are simple and reliable — don't force this into a neural network under time pressure. It's fine to describe this as the system's "decision logic" built on top of the AI-predicted health scores, rather than claiming it's ML when it isn't.
- **Optional stretch, only if round 1 and 2 are solid:** a short natural-language "mission brief" generated per drone (e.g. an LLM call that turns waypoints into a one-line human-readable mission description) is a cheap, visible way to add a second AI touchpoint without new infrastructure.

When judges ask "where's the AI," the honest, confident answer is: "the RUL model predicts each drone's remaining flight time from sensor data, and that prediction feeds directly into the routing decision." That's a true, specific claim — have whoever owns the ML piece ready to say it exactly like that.

GitHub — kept deliberately simple

With 28 hours and basic git experience, the goal is zero merge-conflict drama, not a textbook branching strategy.

Folder structure (one repo, one folder per layer so your changes physically can't collide):

```
drone-airspace-guardian/
  backend/          # simulation engine, API, WebSocket, reroute logic
  frontend/         # Next.js app, map UI
  ml/               # C-MAPSS training script, saved model, OpenSky loader
  README.md         # what it is, how to run it, in 10 lines
  .gitignore        # node_modules/, __pycache__/, .env, *.pkl if large
```

Workflow:

1. One person creates the repo and adds the other two as collaborators immediately.
2. Work directly on `main`, but only inside your own folder — this avoids most conflicts without needing branch management overhead.
3. Commit small and often (every 20–30 minutes, not once every 3 hours) with a one-line message saying what changed, and `git push` right after. A commit you haven't pushed can be lost if your laptop dies.
4. Before you push, run `git pull` first. If Git reports a conflict, it will almost always be in a file outside your folder that you shouldn't have touched — if that happens, stop and message the other two rather than guessing.

5. Only reach for a feature branch if two of you genuinely need to touch the same file at the same time (e.g. the small shared "types" or "API contract" file both frontend and backend read) — for everything else, branches cost more time than they save here.
6. Agree on the API contract (what fields a drone object has, what the WebSocket message looks like) as a five-minute conversation in the first hour, write it down in the README, and don't change field names later without telling both other people — this single step prevents most integration pain.

Keep the README current as you go (what it does, how to run backend/frontend/ml locally, what's still missing) — judges and teammates half-asleep at 3 AM will both thank you.

Minimum criteria checklist

Round 1 (5-8 PM) — must all be true, or the demo has nothing to show:

- ☐ Live map renders with at least 5 simulated drones moving smoothly
- ☐ Each drone has a mission (start point → end point) it's visibly following
- ☐ At least one no-fly zone is drawn and visible on the map
- ☐ An "emergency" trigger exists (a button is fine) that introduces a new danger
- ☐ The system automatically identifies which drones are affected
- ☐ Those drones' paths visibly change on the map in response, without a page reload
- ☐ A visible alert (banner, color change, notification) fires when this happens
- ☐ You can explain, in one sentence each, what the frontend, backend and AI piece do

Round 2 (1-4 AM) — on top of everything above:

- ☐ Each drone shows a predicted health/remaining-flight value from the trained RUL model
- ☐ A drone with low predicted health is automatically flagged and rerouted to land
- ☐ At least a few drones follow paths derived from real OpenSky trajectory data
- ☐ Multiple no-fly zones can exist at once
- ☐ The UI clearly separates "normal operation" from "alert state" at a glance

Round 3 / final (10 AM-1 PM) — polish, not new core features:

- ☐ The full demo has been rehearsed at least twice without a crash
- ☐ A backup screen-recording of a successful run exists in case live demo fails

- ☐ Every team member can answer "where's the AI" and "how does rerouting work" without hesitating
- ☐ The pitch explicitly ties the demo back to the track's stated goals (safety, efficiency, use of aerial/telemetry data)

If you're short on time anywhere, cut features from the bottom of this list upward — never cut anything from the round 1 list, since that's what keeps you in the competition at all.

The demo script

Keep it under 3 minutes of talking, with the emergency moment as the emotional peak. One person narrates, one drives the keyboard, one is ready to answer technical questions — decide who does which before you walk in.

1. **Hook (15 sec):** "As drones scale up around cities and airports, nobody is managing shared airspace the way air traffic control manages planes. We built that missing layer."
2. **Show normal operation (30 sec):** Point at the live map. "Here are [N] drones, each on a real mission, each with a live-predicted health score from a model we trained on turbofan engine degradation data — the same kind of sensor data a drone's motor would produce."
3. **Draw a no-fly zone (15 sec):** "An operator can mark any area — a hospital, an airport — as restricted, live."
4. **Trigger the emergency (the peak, 30–45 sec):** "Now watch — a helicopter enters the airspace." Click it. Narrate what's happening as it happens: "The system just identified [X] affected drones, calculated safe alternate routes, and rerouted them automatically — no human clicked anything for that part."
5. **Show the health-based decision (15 sec, round 2+):** "This drone's predicted remaining flight time dropped below threshold, so it was automatically sent to land instead of continuing its mission — that decision came from the ML model, not a hardcoded rule."
6. **Close (15 sec):** State what you'd build next with more time (e.g. real drone hardware integration, the VisDrone detection data, multi-city zones) to show judges you understand the larger problem, not just this build.

Rehearse this exact script twice before each review, out loud, with the actual app — not a description of what it will do. If something is flaky, cut it from the script rather than hoping it works live.

Judging alignment and fallback plan

How this maps to the track brief. The brief names autonomous navigation, surveillance, mapping, predictive maintenance, and airspace management as the target areas, and stresses that AI is “vital for safety and efficiency.” Your build hits three of those directly: airspace management (the core reroute system), predictive maintenance (the RUL model), and navigation (mission paths, and OpenSky-derived realism). Say those words explicitly in your pitch — judges scoring against a brief notice when a team’s language matches it.

What likely gets scored: working demo over slides, a real AI/ML component over hardcoded logic dressed up as AI, a clear and specific problem statement, and technical depth the team can actually explain. This roadmap is built around all four — don’t trade a working core feature for a flashier but shakier one.

Fallback plan, in order of severity:

- **A feature is behind schedule at a checkpoint:** cut it from the demo script, don’t try to half-show it. A clean 5-drone demo beats a buggy 15-drone one.
- **Live demo breaks during a review:** have a 60–90 second screen recording of a successful run ready on a laptop, recorded during the 4:45 PM or 12:30 AM rehearsal slots. Say plainly “here’s a recording from our last successful run” and keep presenting — judges respect that far more than a team fumbling live for five minutes.
- **Internet/venue Wi-Fi is unreliable:** run everything locally (map tiles cached, backend and frontend on the same laptop) rather than depending on a deployed server judges have to reach over the venue network.
- **One teammate’s piece isn’t ready in time:** the other two should know enough about it (from the shared API contract) to demo a stub or mocked version rather than skip that layer entirely.

The single biggest risk for a 3-person, basic-skill team in 28 hours isn’t a lack of ideas — it’s spending too long on any one piece before integration happens. Get frontend, backend and the WebSocket connection talking to each other, even with fake data, in the first two hours. Everything else — the real ML model, the real datasets, the polish — slots into a pipeline that already works, rather than being bolted onto three pieces that have never been tested together.